FACULTAD DE INFORMATICA | UNIVERSIDAD NACIONAL DE LA PLATA

Orientación a Objetos 2
Cuadernillo Semestral de Actividades
- Refactoring -

Actualizado: 30 de marzo de 2026

El presente cuadernillo estará en elaboración durante el semestre y tendrá un compilado con todos los ejercicios que se usarán durante la asignatura respecto al tema Refactoring.

Recomendación importante:
Los contenidos de la materia se incorporan y fijan mejor cuando uno intenta aplicarlos - no alcanza con ver un ejercicio resuelto por alguien más. Para sacar el máximo provecho de los ejercicios, es importante asistir a las consultas de práctica habiendo intentado resolverlos (tanto como sea posible). De esa manera las consultas estarán más enfocadas y el docente podrá dar un mejor feedback.

Ejercicio 1: Algo huele mal

Indique qué malos olores se presentan en los siguientes ejemplos.

1.1 Protocolo de Cliente

La clase `Cliente` tiene el siguiente protocolo. ¿Cómo puede mejorarlo?

```
/**
* Retorna el límite de crédito del cliente
*/
public double lmtCrdt() {...

/**
* Retorna el monto facturado al cliente desde la fecha f1 a la fecha f2
*/
protected double mtFcE(LocalDate f1, LocalDate f2) {...

/**
* Retorna el monto cobrado al cliente desde la fecha f1 a la fecha f2
*/
private double mtCbE(LocalDate f1, LocalDate f2) {...
```

1.2 Participación en proyectos

Al revisar el siguiente diseño inicial (Figura 1), se decidió realizar un cambio para evitar lo que se consideraba un mal olor. El diseño modificado se muestra en la Figura 2. Indique qué tipo de cambio se realizó y si lo considera apropiado. Justifique su respuesta.

Diseño inicial:

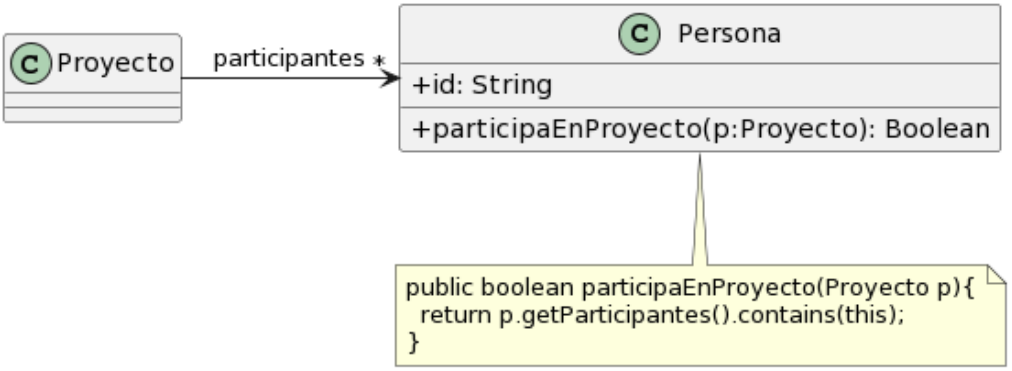


Figura 1: Diagrama de clases del diseño inicial.

Diseño revisado:

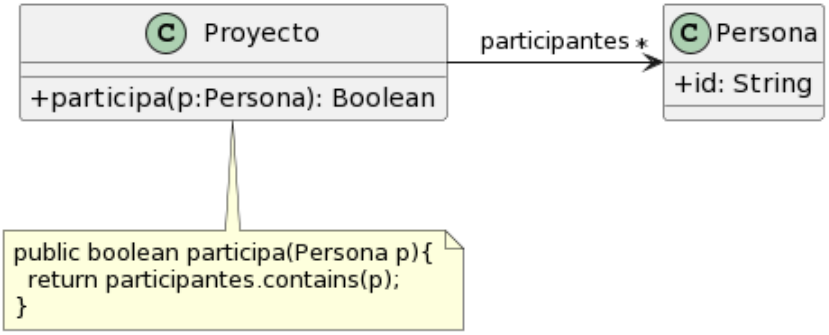


Figura 2: Diagrama de clases modificado.

1.3 Cálculos

Analice el código que se muestra a continuación. Indique qué *code smells* encuentra y cómo pueden corregirse.

```
public void imprimirValores() {
    int totalEdades = 0;
    double promedioEdades = 0;
    double totalSalarios = 0;

    for (Empleado empleado : personal) {
            totalEdades = totalEdades + empleado.getEdad();
            totalSalarios = totalSalarios + empleado.getSalario();
    }
    promedioEdades = totalEdades / personal.size();

    String message = String.format("El promedio de las edades es %s y el total de salarios es %s",
promedioEdades, totalSalarios);

    System.out.println(message);
}
```

Ejercicio 2: Iteradores circulares

Dada la siguiente implementación:

CharRing.java

```
1  public class CharRing extends Object {
2    char[] source;
3    int idx;
4
5    public CharRing(String srcString) {
6       char result;
7       source = new char[srcString.length()];
8       srcString.getChars(0, srcString.length(), source, 0);
9       result = 0;
10       idx = result;
11    }
12
13    public char next() {
14       int result;
15       if (idx >= source.length)
16          idx = 0;
```

```
17        result = idx++;
18        return source[result];
19    }
20 }
```

Tareas:

1) Se quiere aplicar el refactoring *Rename Variable* sobre la variable ***result*** que se usa en la línea 18 con el nuevo nombre ***currentPosition***.
2) ¿Cómo queda el código final y qué inconveniente se podría encontrar?

Ejercicio 3: Iteradores circulares bis

Se cuenta con las siguientes implementaciones de iteradores circulares, las cuales presentan implementaciones similares.

CharRing.java

```
1   public class CharRing {
2       private char[] source;
3       private int idx;
4
5       public CharRing(String src) {
6           source = src.toCharArray();
7           idx = 0;
8       }
9
10      public char next() {
11          if (idx >= source.length)
12              idx = 0;
13          return source[idx++];
14      }
15  }
```

IntRing.java

```
1   public class IntRing {
2       private int[] source;
3       private int idx;
4
5       public IntRing(int[] src) {
6           source = src;
7           idx = 0;
8       }
9
10      public int next() {
11          if (idx >= source.length)
12              idx = 0;
13          return source[idx++];
14      }
15  }
```

Tareas:

1) Diseñe e implemente Test de Unidad para las clases CharRing e IntRing. Asegúrese de que los test pasen.
2) Aplique el refactoring ***Extract Superclass***. Detalle cada uno de los pasos intermedios que son necesarios para poder aplicar correctamente este refactoring.
3) Verifique que los tests definidos en el paso 1 sigan funcionando correctamente.
4) Realice un diagrama de clases UML con el diseño refactorizado.

Ejercicio 4: Alcance en Redes Sociales

Una nueva red social está desarrollando un sistema para modelar perfiles y publicaciones, y medir su alcance como parte del algoritmo de relevancia. Cada publicación acumula reacciones de los usuarios, y cada perfil consolida el alcance de sus publicaciones amplificándolo según si está verificado o no. El código es el siguiente:

Publicacion.java

```java
package redesocial;
public class Publicacion {
  private String texto;
  private int likes;
  public Publicacion(String texto) {
    this.texto = texto;
    this.likes = 0;
  }
  public void darLike() { likes++; }
  public void darDislike() { likes--; }
  private int procesar() {
      return likes * 3;
  }
  public int calcular() {
      return procesar() * 10;
  }
}
```

Perfil.java

```java
package redesocial;
import java.util.ArrayList;
public class Perfil {
  private boolean verificado;
  private ArrayList<Publicacion> publicaciones;
  public Perfil(boolean verificado) {
      this.verificado = verificado;
      this.publicaciones = new ArrayList<>();
  }
  public void agregarPublicacion(Publicacion p) { publicaciones.add(p); }
  private int bonus() { return verificado ? 2 : 1; }
  private int alcanceDePublicaciones() {
      return publicaciones.stream().mapToInt(p -> p.calcular()).sum();
  }
  public int calcular() {
      return alcanceDePublicaciones() * bonus();
  }
}
```

Tarea:

Liste cada uno de los cambios necesarios, indicando archivo y línea afectados, para cada uno de los siguientes refactorings:

1. Rename method: **procesar** (referenciado en línea 11 de Publicacion.java) por **impacto**
2. Rename method: **calcular** (referenciado en línea 14 de Publicacion.java) por **alcance**
3. Rename method: **calcular** (referenciado en línea 15 de Perfil.java) por **alcance**
4. Rename parameter: el parámetro "**p**" del método **agregarPublicacion** (línea 10 de Perfil.java) por **"publicacion"**

Ejercicio 5: Productos

Se cuenta con un sistema que maneja una jerarquía de productos que pueden ser estadías de hoteles o alquileres de autos. En el siguiente material adicional puede descargar un proyecto con las clases implementadas. Observe que el proyecto cuenta con test unitarios que verifican el funcionamiento correcto de la aplicación. A continuación se muestra un extracto del código:

HotelStay.java

```
1  public class HotelStay extends Product {
2      public double cost;
3      private TimePeriod timePeriod;
4      private Hotel hotel;
5
6      public HotelStay(double cost, TimePeriod timePeriod, Hotel hotel) {
7          this.cost = cost;
8          this.timePeriod = timePeriod;
9          this.hotel = hotel;
10     }
11
12     public LocalDate startDate() {
13         return this.timePeriod.start();
14     }
15
16     public LocalDate endDate() {
17         return this.timePeriod.end();
18     }
19
20     public double priceFactor() {
21         return this.cost / this.price();
```

```
    }

    public double price() {
        return this.timePeriod.duration() * this.hotel.nightPrice() * this.hotel.discountRate();
    }
}

```

CarRental.java

```
public class CarRental extends Product {
    public double cost;
    private TimePeriod timePeriod;
    private Company company;

   public CarRental(double cost, TimePeriod timePeriod, Company company) {
        this.cost = cost;
        this.timePeriod = timePeriod;
        this.company = company;
    }

    public LocalDate startDate() {
        return this.timePeriod.start();
    }

    public LocalDate endDate() {
        return this.timePeriod.end();
    }

    public double price() {
        return this.company.price() * this.company.promotionRate();
    }

    public double cost() {
        return this.cost;
    }
}
```

Tareas:

1. La variable “cost” está declarada como pública, lo que rompe el encapsulamiento de la clase. Utilice el refactoring **Encapsulate Field** y describa brevemente los pasos llevados a cabo. Verifique que los tests provistos sigan funcionando. Discuta con un ayudante:

   a. ¿Es correcto modificar alguno de los tests para que el código refactorizado funcione?
   b. En caso de qué el test falle, ¿qué situación está representando este test?
2. Utilice el refactoring **Rename Field** en el método priceFactor(), para que la variable “cost” se pase a llamar “quote”. Verifique que los tests provistos sigan funcionando. Discuta con un ayudante: ¿en este caso, es necesario modificar alguno de los tests para que el código refactorizado funcione?
3. Se quiere aplicar el refactoring **Pull Up Method** para subir los métodos startDate() y endDate() a la superclase Product. ¿Es posible hacerlo en el código anterior? Justifique su respuesta basándose en las precondiciones del refactoring vistas en la teoría y en el libro de Refactoring de Martin Fowler.
4. Mencione y aplique los refactorings necesarios para poder hacer Pull Up Method.
5. Aplique Pull Up Method para subir los métodos a la superclase Product. Verifique que los tests provistos sigan funcionando.
6. Observe los métodos price() en CarRental.java (línea 21) y en HotelStay.java (línea 25). Identifique los Code Smell presentes. Luego aplique los refactoring correspondientes para eliminarlos. Verifique que los tests provistos sigan funcionando.

Ejercicio 6:

Para cada una de las siguientes situaciones, realice en forma iterativa los siguientes pasos:
(i) indique el mal olor,
(ii) indique el refactoring que lo corrige,
(iii) aplique el refactoring, mostrando el resultado final (código y/o diseño según corresponda).
Si vuelve a encontrar un mal olor, retorne al paso (i).

6.1 Empleados

```
public class EmpleadoTemporario {
   public String nombre;
   public String apellido;
   public double sueldoBasico = 0;
   public double horasTrabajadas = 0;
   public int cantidadHijos = 0;
   // ......

      public double sueldo() {
            return this.sueldoBasico
                        +   (this.horasTrabajadas * 500)
                        +   (this.cantidadHijos * 1000)
                        -   (this.sueldoBasico * 0.13);
      }
}
```

```
public class EmpleadoPlanta {
    public String nombre;
    public String apellido;
    public double sueldoBasico = 0;
    public int cantidadHijos = 0;
    // ......

    public double sueldo() {
        return this.sueldoBasico
                    + (this.cantidadHijos * 2000)
                    - (this.sueldoBasico * 0.13);
    }
}

public class EmpleadoPasante {
    public String nombre;
    public String apellido;
    public double sueldoBasico = 0;
    // ......

    public double sueldo() {
        return this.sueldoBasico - (this.sueldoBasico * 0.13);
    }
}
```

6.2 Juego

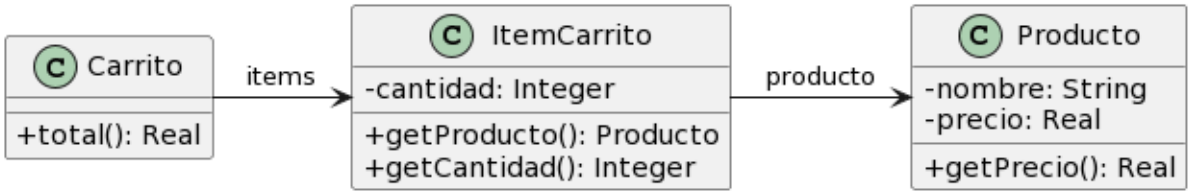
```
public class Juego {
    // ......
    public void incrementar(Jugador j) {
        j.puntuacion = j.puntuacion + 100;
    }
    public void decrementar(Jugador j) {
        j.puntuacion = j.puntuacion - 50;
    }
}
```

```
public class Jugador {
    public String nombre;
    public String apellido;
    public int puntuacion = 0;
}
```

6.3 Publicaciones

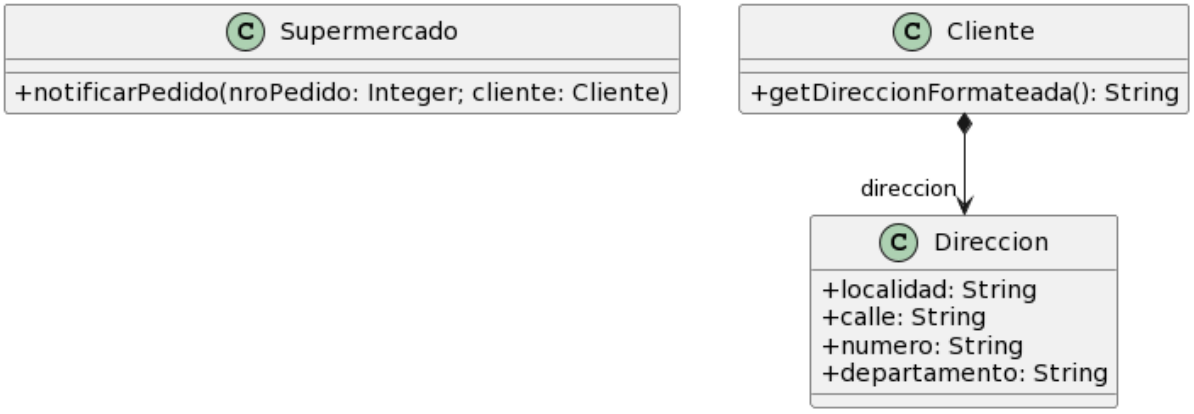
```
/**
* Retorna los últimos N posts que no pertenecen al usuario user
*/
public List<Post> ultimosPosts(Usuario user, int cantidad) {

    List<Post> postsOtrosUsuarios = new ArrayList<Post>();
    for (Post post : this.posts) {
        if (!post.getUsuario().equals(user)) {
            postsOtrosUsuarios.add(post);
        }
    }

   // ordena los posts por fecha
   for (int i = 0; i < postsOtrosUsuarios.size(); i++) {
      int masNuevo = i;
      for(int j= i +1; j < postsOtrosUsuarios.size(); j++) {
          if (postsOtrosUsuarios.get(j).getFecha().isAfter(
     postsOtrosUsuarios.get(masNuevo).getFecha())) {
             masNuevo = j;
          }
      }
      Post unPost = postsOtrosUsuarios.set(i,postsOtrosUsuarios.get(masNuevo));
      postsOtrosUsuarios.set(masNuevo, unPost);
   }

    List<Post> ultimosPosts = new ArrayList<Post>();
    int index = 0;
    Iterator<Post> postIterator = postsOtrosUsuarios.iterator();
    while (postIterator.hasNext() &&  index < cantidad) {
        ultimosPosts.add(postIterator.next());
    }
    return ultimosPosts;
}
```

6.4 Carrito de compras



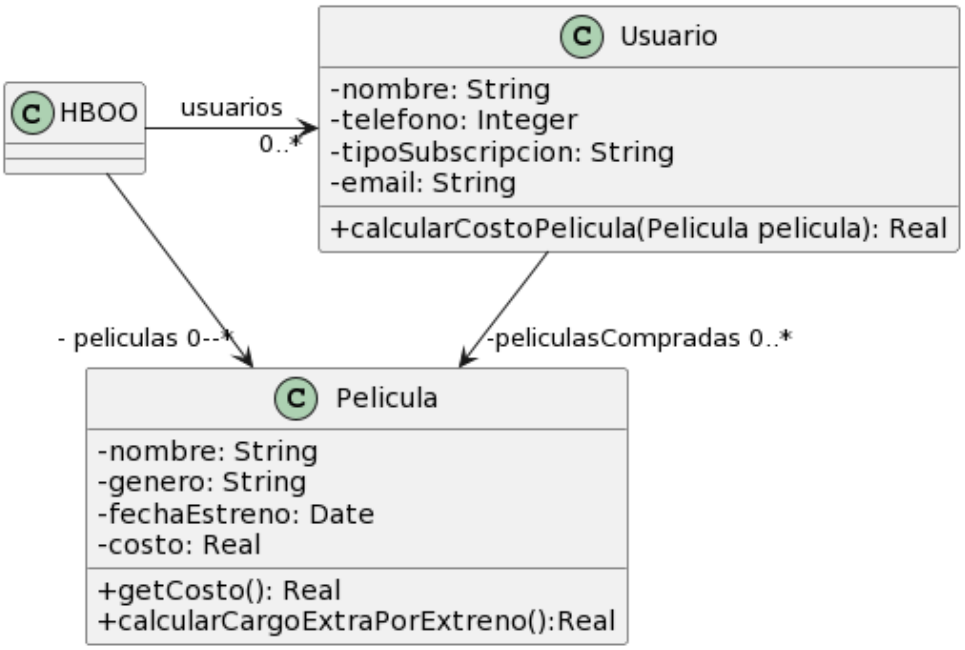

```
public class Producto {
    private String nombre;
    private double precio;

    public double getPrecio() {
        return this.precio;
    }
}

public class ItemCarrito {
    private Producto producto;
    private int cantidad;

    public Producto getProducto() {
        return this.producto;
    }

    public int getCantidad() {
        return this.cantidad;
    }

}

public class Carrito {
    private List<ItemCarrito> items;

    public double total() {
            return this.items.stream()
                    .mapToDouble(item ->
                            item.getProducto().getPrecio() * item.getCantidad())
                    .sum();
    }
}
```

6.5 Envío de pedidos




```
public class Supermercado {
  public void notificarPedido(long nroPedido, Cliente cliente) {
    String notificacion = MessageFormat.format("Estimado cliente, se le informa que hemos recibido su pedido con número {0}, el cual será enviado a la dirección {1}", new Object[] { nroPedido, cliente.getDireccionFormateada() });

    // lo imprimimos en pantalla, podría ser un mail, SMS, etc..
    System.out.println(notificacion);
  }
}

public class Cliente {
  public String getDireccionFormateada() {
    return
            this.direccion.getLocalidad() + ", " +
            this.direccion.getCalle() + ", " +
            this.direccion.getNumero() + ", " +
            this.direccion.getDepartamento()
    ;
}
```

6.6 Películas




```
public class Usuario {
   String tipoSubscripcion;
   // ...

   public void setTipoSubscripcion(String unTipo) {
      this.tipoSubscripcion = unTipo;
   }

   public double calcularCostoPelicula(Pelicula pelicula) {
      double costo = 0;
      if (tipoSubscripcion=="Basico") {
            costo = pelicula.getCosto() + pelicula.calcularCargoExtraPorEstreno();
      }
      else if (tipoSubscripcion== "Familia") {
            costo = (pelicula.getCosto() + pelicula.calcularCargoExtraPorEstreno()) * 0.90;
      }
      else if (tipoSubscripcion=="Plus") {
            costo = pelicula.getCosto();
      }
      else if (tipoSubscripcion=="Premium") {
            costo = pelicula.getCosto() * 0.75;
      }
      return costo;
   }
}

public class Pelicula {
   LocalDate fechaEstreno;
   // ...

   public double getCosto() {
      return this.costo;
```

```
    }

    public double calcularCargoExtraPorEstreno(){
      // Si la Película se estrenó 30 días antes de la fecha actual, retorna un cargo de 0$, caso
contrario, retorna un cargo extra de 300$
      return (ChronoUnit.DAYS.between(this.fechaEstreno, LocalDate.now()) ) > 30 ? 0 : 300;
    }
}
```

Ejercicio 7: Etiquetas

Observe el siguiente código:

```
abstract class Etiqueta {
    protected String nombreProducto;
    protected double precio;

    public Etiqueta(String nombre, double precio) {
        this.nombreProducto = nombre;
        this.precio = precio;
    }
}

class EtiquetaSimple extends Etiqueta {
    public EtiquetaSimple(String nombre, double precio) {
        super(nombre, precio);
    }

    public void generar() {
        System.out.println("--- ETIQUETA BÁSICA ---");
        System.out.println("Producto: " + nombreProducto);
        System.out.println("Precio: $" + precio);
        System.out.println("----------------------");
    }
}

class EtiquetaDetalle extends Etiqueta {
    public EtiquetaDetalle(String nombre, double precio) {
        super(nombre, precio);
    }

    public void generar() {
        System.out.println("--- ETIQUETA DETALLE ---");
        System.out.println("Producto: " + nombreProducto);
        System.out.println("Precio sin imp.: $" + (precio * 0.79));
        System.out.println("Precio final: $" + precio);
        System.out.println("----------------------");
    }
}
```

Tareas:

1. ¿Hay código duplicado? Indique claramente en qué líneas se encuentra.
2. Se quiere aplicar el refactoring *Pull Up Method* para subir el método generar() a la superclase Etiqueta. ¿Es posible hacerlo en el código anterior? Justifique su respuesta basándose en las precondiciones del refactoring vistas en la teoría y en el libro de Refactoring de Martin Fowler.

3. Mencione los refactorings previos necesarios para que sea posible aplicar Pull Up Method.
4. Aplique Pull Up Method para subir el método generar() a la superclase Etiqueta.

Ejercicio 8 - Documentos y estadísticas

Dado el siguiente código implementado en la clase Document y que calcula algunas estadísticas del mismo:

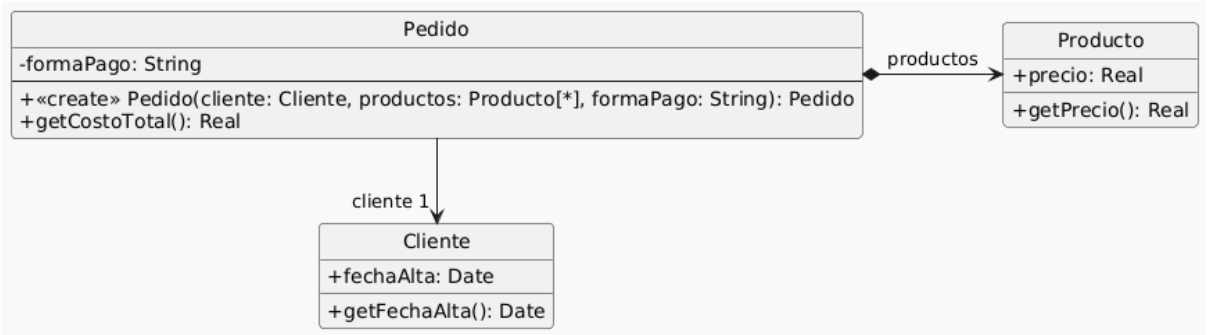
```
public class Document {
   List<String> words;

   public long characterCount() {
         long count = this.words
                  .stream()
                  .mapToLong(w -> w.length())
                  .sum();
         return count;
         }
   public long calculateAvg() {
         long avgLength = this.words
                  .stream()
                  .mapToLong(w -> w.length())
                  .sum() / this.words.size();
         return avgLength;
         }
// Resto del código que no importa
}
```

Tareas:

1. Enumere los code smell y que refactorings utilizará para solucionarlos.
2. Aplique los refactorings encontrados, mostrando el código refactorizado luego de aplicar cada uno.
3. Analice el código original y detecte si existe un problema al calcular las estadísticas. Explique cuál es el error y en qué casos se da ¿El error identificado sigue presente luego de realizar los refactorings? En caso de que no esté presente, ¿en qué momento se resolvió? De acuerdo a lo visto en la teoría, ¿podemos considerar esto un refactoring?

Ejercicio 9 - Pedidos

Se tiene el siguiente modelo de un sistema de pedidos y la correspondiente implementación.




```
01: public class Pedido {
02:  private Cliente cliente;
03:  private List<Producto> productos;
04:  private String formaPago;
05:  public Pedido(Cliente cliente, List<Producto> productos, String formaPago)
{
06:     if (!"efectivo".equals(formaPago)
07:        && !"6 cuotas".equals(formaPago)
08:        && !"12 cuotas".equals(formaPago)) {
09:          throw new Error("Forma de pago incorrecta");
10:    }
11:    this.cliente = cliente;
12:    this.productos = productos;
13:    this.formaPago = formaPago;
14:   }
15:   public double getCostoTotal() {
16:     double costoProductos = 0;
17:     for (Producto producto : this.productos) {
18:       costoProductos += producto.getPrecio();
19:     }
20:     double extraFormaPago = 0;
21:     if ("efectivo".equals(this.formaPago)) {
22:       extraFormaPago = 0;
23:     } else if ("6 cuotas".equals(this.formaPago)) {
24:       extraFormaPago = costoProductos * 0.2;
25:     } else if ("12 cuotas".equals(this.formaPago)) {
26:       extraFormaPago = costoProductos * 0.5;
27:     }
28:     int añosDesdeFechaAlta = Period.between(this.cliente.getFechaAlta(),
LocalDate.now()).getYears();
29:     // Aplicar descuento del 10% si el cliente tiene más de 5 años de
antiguedad
30:     if (añosDesdeFechaAlta > 5) {
31:       return (costoProductos + extraFormaPago) * 0.9;
32:     }
33:     return costoProductos + extraFormaPago;
34:   }
35: }
36: public class Cliente {
37:   private LocalDate fechaAlta;
38:   public LocalDate getFechaAlta() {
39:     return this.fechaAlta;
40:   }
```

```
41: }
42: public class Producto {
43:   private double precio;
44:   public double getPrecio() {
45:     return this.precio;
46:   }
47: }
```

Tareas:

1. Dado el código anterior, aplique **únicamente** los siguientes refactoring:
    - Replace Loop with Pipeline (líneas 16 a 19)
    - Replace Conditional with Polymorphism (líneas 21 a 27)
    - Extract method y move method (línea 28)
    - Extract method y replace temp with query (líneas 28 a 33)
2. Realice el diagrama de clases del código refactorizado.